

Planes de entornos para LOCA

Cómo profesionalizar la arquitectura de deploy: del commit directo a `main` hacia un pipeline con validación previa a producción.

Stack: Next.js · Vercel · Supabase · Documento de trabajo · 25/08/2026

Contexto y objetivo

Hoy los cambios se commitean directo a `main`, que Vercel deploya automáticamente a producción. No hay ningún paso de validación entre "escribo el código" y "está online para los clientes". El objetivo es introducir uno o más entornos intermedios para que **ningún cambio llegue a producción sin haber corrido antes en un entorno equivalente**, con datos que no son los reales.

El estándar de la industria es la progresión `local` → `preview` → `staging` → `production`. Este documento describe dos planes: uno mínimo que se puede adoptar hoy sin infraestructura nueva (Plan A), y uno completo con un staging persistente (Plan B). Son **complementarios y secuenciales**: el Plan A es el primer escalón del Plan B.

El matiz que más importa

Un entorno de prueba que apunta a la **misma base de datos** y a las **mismas apps de Meta/Instagram** que producción no es un entorno de prueba: es producción con otra URL. Una prueba de OAuth escribiría tokens reales y un "Publicar" postearía de verdad en las redes de un cliente. El valor real de staging está en **aislar los datos y las integraciones**, no en tener otra URL.

Plan A — Ramas + Preview deployments

Bajo costo · Adoptable hoy

Aprovecha los **Preview deployments** que Vercel ya provee gratis: cada Pull Request levanta automáticamente su propia URL temporal con ese cambio aislado. No requiere infraestructura nueva. El objetivo es dejar de commitear directo a `main`.

Qué se gana

- Cada cambio se revisa en una URL propia antes de tocar producción.
- `main` queda protegido: solo entra código ya revisado y mergeado por PR.
- Historial limpio y trazable (qué cambió, quién lo aprobó, cuándo salió).
- Rollback trivial: revertir el merge.

Limitaciones

- El preview **comparte la base de datos y las apps de Meta de producción** por defecto (mismas env vars). Sirve para revisar UI y lógica, no para probar a fondo integraciones que escriben datos reales.
- Es efímero (una URL por PR), no un entorno estable donde hacer QA integral o mostrarle a un tercero.

Pasos de implementación

1. **Convención de ramas.** Trabajar en ramas por feature (`feat/instagram-login` , `fix/...`) en vez de sobre `main` .

2. **Pull Requests.** Abrir un PR hacia `main` por cada cambio. Vercel comenta la URL de preview en el PR automáticamente.
3. **Branch protection en GitHub.** Proteger `main` : exigir PR (no push directo), y opcionalmente que el build de Vercel pase antes de poder mergear.
4. **Revisión + merge.** Se valida en la URL de preview; al mergear, Vercel deploya `main` a producción.

Esfuerzo estimado: bajo (configuración de un par de horas, sin costos nuevos).

Plan B — Staging persistente

Costo medio · Infra nueva

Un entorno **estable y fijo** en `stg.app.heyloca.ai` que refleja "lo próximo que sale a producción". A diferencia del preview efímero, es donde se hace QA integral, se prueban las integraciones con terceros de punta a punta, y se valida antes de promover a producción. Incluye el Plan A como base.

Arquitectura por entorno

La clave es **duplicar el stack**, no solo el dominio. Cada entorno tiene su propio par de todo:

Componente	Producción	Staging
Dominio	<code>app.heyloca.ai</code>	<code>stg.app.heyloca.ai</code>
Rama Git	<code>main</code>	<code>staging</code>
Base de datos	Proyecto Supabase prod	Proyecto Supabase staging (migraciones propias)
Apps Meta / IG	App real (App Review)	App de test, o redirects/webhooks a <code>stg.app.</code>
Claves / secrets	Set de producción	Set propio (incl. <code>META_TOKEN_ENCRYPTION_KEY</code>)
Datos	Clientes reales	Cuentas de prueba, datos descartables

Flujo de trabajo

`feat/*` → PR → **preview** (Plan A) → merge a `staging` → **stg.app.heyloca.ai** (QA) → promoción a `main` → **producción**

Pasos de implementación

- Base de datos de staging.** Crear un segundo proyecto en Supabase y aplicarle todas las migraciones (`0001` ... `0007`). Es lo que garantiza que una prueba no toque datos reales.
- Rama y dominio.** Crear la rama `staging` y en Vercel asignar el dominio `stg.app.heyloca.ai` a esa rama (Git branch → domain).
- Env vars por entorno.** Vercel permite alcance `Production` / `Preview` por variable. Cargar el set de staging (Supabase staging, claves propias, redirect URIs de `stg.app.`) separado del de producción.
- Integraciones aisladas.** Apps de test de Meta/Instagram/LinkedIn, o como mínimo redirect URIs y webhooks apuntando a `stg.app.heyloca.ai` . Sin esto, un OAuth de staging escribe en la conexión real y un "Publicar" postea de verdad.
- Promoción a producción.** Merge `staging` → `main` una vez validado. `main` sigue deployando a producción como hoy.
- Datos de prueba.** Sembrar staging con cuentas y negocios de prueba; nunca copiar datos de clientes reales sin anonimizar.

Riesgo específico del momento (App Review de Meta)

Mientras la app de Meta esté **sin publicar**, solo admins/devs/testers pueden conectar cuentas. Un staging que use la **misma** app de Meta hereda esa limitación y, peor, mezcla pruebas con la app que estás por mandar a revisión. Lo ideal es una **app de test separada** para staging, dejando la app real intacta para el App Review en curso.

Costo real

El costo no es el dominio ni Vercel (ambos baratos o incluidos). El costo es **mantener el segundo Supabase y las apps de Meta duplicadas**: aplicar cada migración en dos lados, mantener dos sets de secrets, y sembrar datos de prueba. Es un costo recurrente de mantenimiento, no un setup de una sola vez.

Esfuerzo estimado: medio (setup de 1–2 días + mantenimiento continuo).

Comparación rápida

Criterio	Plan A — Preview	Plan B — Staging
Infraestructura nueva	Ninguna	Supabase + apps Meta de staging
Costo	Incluido en Vercel	Medio (mantenimiento recurrente)
Aísla datos reales	No (comparte DB de prod)	Sí
Prueba integraciones a fondo	Limitado	Sí, de punta a punta
Entorno estable para QA / demos	No (efímero por PR)	Sí (URL fija)
Protege <code>main</code>	Sí	Sí
Adoptable hoy	Sí	Requiere setup previo

Recomendación

Ir por fases, no montar todo de una:

- **Ahora:** adoptar el **Plan A**. Saca el hábito de commitear directo a `main` y da una URL de prueba por cambio, sin infraestructura ni costos nuevos. Es el primer escalón y aplica ya.
- **Cuando haya tiempo de infra:** subir al **Plan B** con el staging persistente. Priorizarlo *después* del App Review de Meta, para no interferir con la app que está en revisión — o directamente arrancarlo con una app de test separada desde el día uno.